

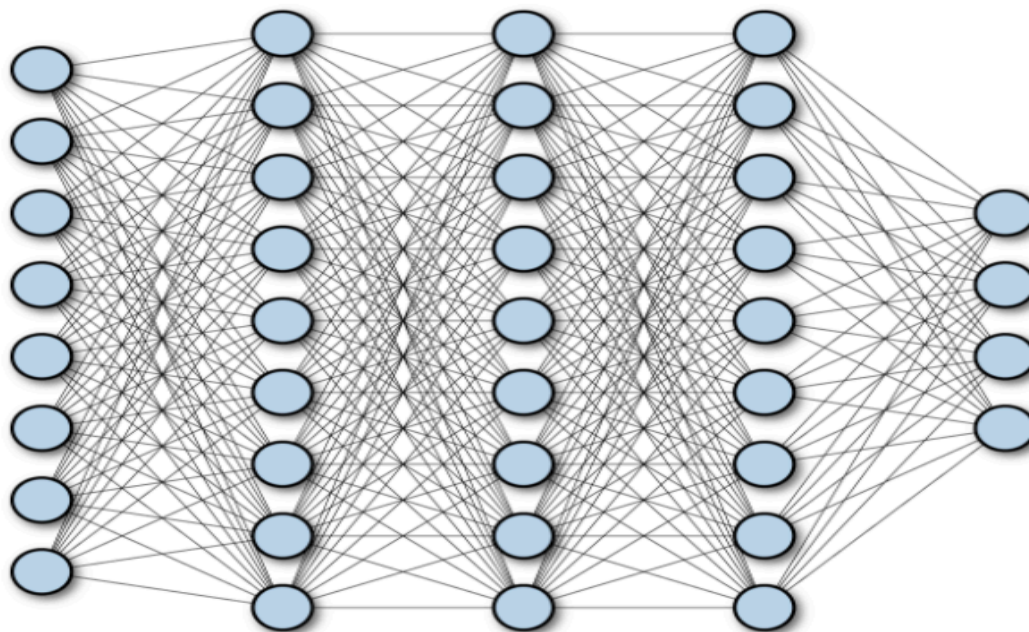


مهلت تحویل: سه شنبه 9 تیر ماه 1405، ساعت 23:59

مقدمه

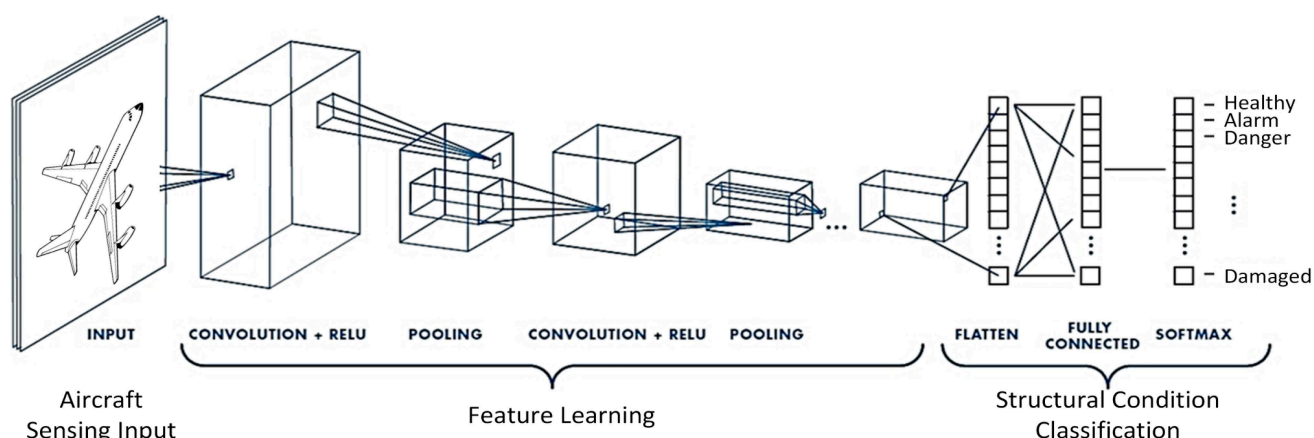
شبکه‌های عصبی Fully Connected

در شبکه‌های عصبی Fully Connected که در درس نیز با آن آشنا شدید، هر تصویر ابتدا مسطح شده و به صورت بردار به عنوان ورودی شبکه داده می‌شود. هر درایه این بردار (معادل با یک پیکسل تصویر) یک ویژگی برای آن محسوب می‌شود. شبکه قرار است بر اساس این ویژگی‌ها و با ساختن ترکیبات غیرخطی از آن‌ها، وزن اتصالات بین لایه‌هایش را طوری تنظیم کند، که خروجی آن ضمن داشتن کمترین خطا، کلاس تصویر ورودی متناظر را به درستی پیش‌بینی کند. شکل زیر ساختار کل این نوع از شبکه‌ها را نشان می‌دهد:



شبکه‌های عصبی پیچشی

شبکه‌های عصبی پیچشی (Convolutional Neural Networks) نوعی مدل یادگیری عمیق هستند که به طور گسترده و به خصوص برای پردازش تصویر و ویدیو استفاده می‌شوند (شکل ۱). این مدل‌ها برای تجزیه و تحلیل و شناسایی موثر الگوهای بصری با استفاده از مفهوم Convolution طراحی شده‌اند. CNN ها معمولاً از چندین لایه شامل لایه‌های پیچشی^۱، لایه‌های ادغام^۲ و لایه‌های کاملاً متصل^۳ تشکیل می‌شوند. لایه‌های کانولوشن، فیلترهایی را روی تصویر ورودی اعمال می‌کنند و ویژگی‌ها را در مکان‌های مختلف تصویر یا ویدیو استخراج می‌کنند. لایه‌های ادغام ابعاد فضایی ویژگی‌ها را کاهش می‌دهند و لایه‌های کاملاً متصل بر اساس ویژگی‌های استخراج شده، عملیات طبقه‌بندی و یا رگرسیون مورد نظر را انجام می‌دهند. این معماری سلسله مراتبی، CNN ها را قادر می‌سازد تا الگوهای بصری پیچیده را به طور خودکار یاد بگیرند. این قابلیت مدل‌های پیچشی، آن‌ها را در کارهایی مانند طبقه‌بندی تصویر، تشخیص اشیا^۴ و بخش‌بندی تصویر^۵ بسیار پر قدرت می‌سازد. تصویر زیر ساختار کلی یک شبکه عصبی پیچشی را نشان می‌دهد:



Convolutional Layers¹

Pooling Layers²

Fully Connected Layers³

Object Detection⁴

Image Segmentation⁵

تعریف مسئله

در این تمرین، شما ابتدا با استفاده از یک شبکه عصبی fully connected و سپس با استفاده از یک شبکه عصبی پیچشی به طبقه‌بندی تصاویر با استفاده از کتابخانه **PyTorch** می‌پردازید و در نهایت نتایج حاصل از این دو شبکه را با یکدیگر مقایسه می‌کنید. پیشنهاد می‌شود قبل از انجام پروژه، نوتبوک مربوط به آشنایی با **PyTorch** را مطالعه کنید تا آشنایی لازم با این کتابخانه را بدست آورید. همچنین برای انجام پروژه از نوتبوک نیمه آماده ای که در اختیارتان قرار گرفته است استفاده کنید.

برای راحتی استفاده از کتابخانه‌ها و تسریع فرایند آموزش، توصیه می‌شود که از سرویس **Google Colab** استفاده کنید. توجه کنید که آموزش شبکه‌های عمیق (به خصوص شبکه‌های CNN) بر روی GPU بسیار سریع‌تر از CPU می‌باشد. در نتیجه توصیه می‌شود که در صورت استفاده از سرویس **Google Colab** نوع **Runtime Type** را بر روی GPU قرار دهید. دقت کنید که مدت زمان استفاده از GPU به ازای هر اکانت محدود می‌باشد، در نتیجه فقط در هنگام آموزش مدل از GPU استفاده کنید تا مدت زمان اختصاص یافته برای شما به اتمام نرسد.

آشنایی با مجموعه داده

مجموعه داده مورد استفاده در این تمرین **Blood Cell Images for Cancer Detection** است. این مجموعه داده یکی از مجموعه‌های پرکاربرد در حوزه تحلیل تصاویر پزشکی و بینایی کامپیوتر است که برای آموزش و ارزیابی سیستم‌های تشخیص خودکار لوسمی (سرطان خون) مورد استفاده قرار می‌گیرد. این مجموعه شامل 5,000 تصویر رنگی با وضوح بالا است که به 5 کلاس مختلف تقسیم‌بندی شده‌اند و هر کلاس شامل 1,000 تصویر است. این 5 کلاس عبارتند از:

- basophil
- erythroblast
- monocyte
- myeloblast
- seg_neutrophil

برای اطلاعات بیشتر در مورد این مجموعه داده می‌توانید به این [لینک](#) مراجعه کنید.

پیش‌پردازش داده‌ها

در این بخش، پیش از آنکه بتوانید مدل‌های شبکه عصبی خود را آموزش دهید، باید تغییراتی را روی تصاویر اعمال کنید تا برای ورود به شبکه و انجام محاسبات بهینه مناسب باشند.

ابتدا مجموعه داده‌ی ۵۰۰۰ تایی سلول‌های خونی را که در اختیار دارید، به دو بخش آموزش^۶ و تست^۷ تقسیم کنید.

در گام بعدی، با استفاده از ابزارهای پیش‌پردازش تصویر موجود در PyTorch، عملیات زیر را روی تمامی داده‌ها اعمال کنید:

- تغییر اندازه^۸: از آنجا که تصاویر اصلی دارای وضوح بالایی هستند و پردازش آن‌ها زمان‌بر و نیازمند حافظه زیادی است، ابعاد تمام تصاویر را به 64x64 پیکسل کاهش دهید.
- نرمال‌سازی^۹: مقادیر پیکسلی تصاویر را که در حالت عادی در بازه 0 تا 255 قرار دارند، به یک فرمت استاندارد نرمال‌سازی کنید.

پس از اعمال این تغییرات، داده‌های آماده‌شده را به فرمت DataLoader تبدیل کنید.

- درباره اهمیت نرمال‌سازی داده‌ها در آموزش شبکه‌های عصبی عمیق تحقیق کنید. اگر مقادیر ورودی را پیش از تغذیه به شبکه نرمال نکنیم، چه تاثیری بر روند همگرایی مدل و الگوریتم گرادیان کاهشی خواهد داشت؟

- چرا از همان فرمت Dataset استفاده نمی‌کنیم و آن را به فرمت DataLoader در می‌آوریم؟
- DataLoader چه مزیتی نسبت به فرمت Dataset دارد؟
- برای آشنایی بهتر با مجموعه داده از هر کلاس 5 تصویر قبل از نرمال‌سازی را به صورت رندوم نشان دهید.

بخش اول: طبقه‌بندی با استفاده از شبکه عصبی Fully Connected

در این بخش از یک شبکه عصبی fully connected برای طبقه‌بندی تصاویر استفاده می‌کنید. ساختار این شبکه عصبی را در قالب کلاس BloodCellMLP پیاده‌سازی کنید. این کلاس باید از nn.Module ارث‌بری کند؛ در واقع تمام مدل‌های PyTorch باید از این کلاس پایه مشتق شوند تا بتوانند از قابلیت‌هایی مثل ثبت پارامترها ذخیره و بارگذاری وزن‌ها و ... استفاده کنند.

^۶ Train

^۷ Test

^۸ Resize

^۹ Normalization

همانطور که در بخش پیش پردازش مشخص شد، هر تصویر پس از تغییر اندازه دارای ابعاد 64x64 می‌شود و ۳ کانال رنگی دارند. از آنجا که در این نوع از شبکه‌ها تصاویر به صورت بردار به عنوان ورودی داده می‌شوند، ابعاد ورودی را متناسب با پیکسل‌های تصویر مسطح‌شده در نظر بگیرید. همچنین تعداد کلاسهای نهایی برابر 5 تا می‌باشد.

سپس به سراغ تعریف معماری این شبکه عصبی بروید. در طراحی شبکه خود می‌توانید از dropout استفاده کنید. این روش یکی از روشهای کارا برای کاهش بیش‌برازش¹⁰ در شبکه‌های عصبی است.

حال یک instance از مدلی که ساخته شده ایجاد کنید و آن را به device ای (GPU یا CPU) که می‌خواهید مدل را روی آن آموزش دهید منتقل کنید. سپس نیاز دارید که یک تابع هزینه و یک الگوریتم بهینه سازی برای مدل خود انتخاب کنید. بهینه سازهای مختلفی برای وزن ها و پارامترهای شبکه‌های عصبی وجود دارد یکی از معروف ترین بهینه سازهای موجود بهینه ساز Adam است.

حال فرایند آموزش مدل را شروع کنید. در طی این فرایند باید موارد زیر برای هر epoch نشان داده شوند:

- Number of Epoch
- Train Loss
- Test Loss

پس از اتمام فرایند آموزش مقادیر Train Loss و Test Loss را به ازای هر epoch ذخیره کنید تا در ادامه بتوانید نمودارهای مربوطه را برای تحلیل مدل رسم کنید و مقایسه درستی از مدل ها داشته باشید. توجه همانطور که گفته شد برای مقایسه این دو مدل باید تا حد امکان مواردی که بین دو مدل مشترک است را یکسان در نظر گرفت تا مقایسه این دو مدل تا حد امکان در شرایط یکسانی صورت گیرد. یکی از این موارد، تعداد epoch است. این تعداد را برای مدل خود 80 در نظر بگیرید. همچنین حداکثر تعداد پارامترهای مدل‌ها در حدود ۳.۵ میلیون پارامتر در نظر گرفته شود. حداقل دقت¹¹ برای دریافت نمره کامل این بخش ۵۵ درصد می‌باشد.

حال با استفاده از مقادیر ذخیره شده نمودار Loss را بر اساس شماره epoch ها رسم کنید. در نهایت مدل خود را بر روی داده تست ارزیابی کنید. برای ارزیابی مدل مقادیر confusion matrix و classification report را محاسبه و رسم کنید.

¹⁰ Overfitting

¹¹ Accuracy

- درباره نحوه عملکرد dropout و تاثیر آن روی مدل تحقیق کنید.
- بر اساس معماری طراحی شده برای این مدل، تعداد پارامترهای مدل را به صورت دستی محاسبه کرده و نحوه محاسبه را ذکر کنید و در نهایت آن را با نتیجه کد خود در نوتبوک مقایسه کنید.
- درباره تابع هزینه Cross Entropy تحقیق کنید و دلیل استفاده از آن را با توجه به ماهیت مسئله توضیح دهید.
- درباره نحوه کارکرد بهینه‌ساز Adam تحقیق کنید. چه تفاوت‌های اصلی ای بین Adam و SGD که در درس با آن آشنا شدید وجود دارد؟
- نمودار فرآیند یادگیری (Loss Curve) رسم‌شده را تحلیل کنید. آیا بیش برآزش رخ داده است؟ چگونه میتوان این موضوع را فهمید؟

بخش دوم: طبقه‌بندی با استفاده از شبکه عصبی پیچشی (CNN)

در این بخش هدف ما حل همان مسئله ی طبقه‌بندی مجموعه داده‌ی Blood Cell با استفاده از یک شبکه ی عصبی پیچشی Convolutional Neural Network یا (CNN) است. سپس عملکرد این مدل را با شبکه‌ی عصبی Fully Connected که در بخش اول پیاده سازی کردید مقایسه خواهیم کرد هدف نهایی درک بهتر تفاوت معماری و توانایی‌های یادگیری این دو نوع شبکه است.

1. آموزش و ارزیابی شبکه عصبی پیچشی: در ادامه‌ی مسیر قبلی، اکنون باید یک شبکه‌ی عصبی پیچشی طراحی و آموزش دهید تا بتوانید عملکرد آن را با مدل Fully Connected مقایسه کنید. برای این منظور، ابتدا ساختار شبکه را در قالب کلاسی با نام CNN پیاده‌سازی می‌کنید. این مدل باید به‌گونه‌ای طراحی شود که تعداد پارامترهای آموزش‌پذیر آن تقریباً برابر تعداد پارامترهای مدل Fully Connected باشد تا شرایط مقایسه منصفانه با مدل قبلی فراهم شود.

پس از طراحی معماری شبکه، لازم است تعداد کل پارامترهای آن را به صورت دستی محاسبه کرده و نحوه ی محاسبه را به وضوح بیان کنید. سپس این مقدار را با عدد به دست آمده از اجرای کد در نوت‌بوک مقایسه کنید تا از صحت طراحی خود اطمینان حاصل نمایید.

پس از تعریف موارد فوق، مدل را به مدت epoch 80 آموزش دهید. در هر دوره، مقادیر Loss را برای داده‌های آموزش و تست محاسبه، ذخیره و نمایش دهید. در نهایت مدل خود را بر روی داده تست ارزیابی کنید. برای ارزیابی مدل مقادیر confusion matrix و classification report را محاسبه و رسم کنید. حداقل دقت این مدل برای دریافت نمره کامل ۸۵ درصد است. همچنین محدودیت ۳.۵ میلیون پارامتری برای مقایسه دو مدل باید روی این مدل هم اعمال شود.

در مرحله‌ی بعد، مشابه بخش قبل، باید تابع هزینه و الگوریتم بهینه‌سازی مناسبی انتخاب شود.

- ابعاد فیلتر (Kernel Size) چه تاثیری بر استخراج ویژگی‌ها در شبکه‌های پیچشی دارد؟ استفاده از کرنل‌های کوچکتر در مقایسه با کرنل‌های بزرگتر چه مزایا و معایبی از نظر قدرت تشخیص الگوهای محلی (Local Features) و همچنین بار محاسباتی مدل دارد؟
- افزایش عمق شبکه (تعداد لایه‌های پیچشی متوالی) چه تاثیری بر ظرفیت یادگیری، درک ویژگی‌های انتزاعی و عملکرد نهایی مدل دارد؟ آیا عمیق‌تر کردن بیش از حد شبکه همواره باعث بهبود دقت می‌شود؟ پدیده‌هایی مانند محو شدن گرادیان¹² و بیش‌برازش¹³ در شبکه‌های بسیار عمیق را در این زمینه تحلیل کنید.

بخش سوم: بررسی خطاهای مدل و تحلیل خروجی Softmax

برای درک بهتر روند تصمیم‌گیری مدل‌ها، بررسی نمونه‌هایی که شبکه به اشتباه پیش‌بینی کرده است اهمیت بالایی دارد. با تحلیل خروجی‌های تابع Softmax، می‌توانیم مشخص کنیم که آیا شبکه در انتخاب اشتباه خود کاملاً مطمئن بوده یا اینکه در تفکیک کلاس‌های مشابه دچار تردید شده است. این بینش برای بررسی نقاط ضعف مدل بسیار مفید است.

از شما خواسته می‌شود برای هر یک از مدل‌های آموزش‌دیده (MLP و CNN)، تعداد ۵ تصویر از مجموعه داده تست را که مدل به صورت نادرست طبقه‌بندی کرده است پیدا کرده و نمایش دهید. برای هر تصویر باید برچسب پیش‌بینی شده و برچسب واقعی ذکر شود. در صورتی که تصاویر را پیش‌تر نرمال‌سازی کرده اید پیش از نمایش باید آنها را unnormalized کنید تا رنگ‌ها و ظاهر آن‌ها طبیعی و قابل تفسیر باشند.

علاوه بر نمایش تصویر، باید توزیع احتمالات خروجی شبکه (پس از اعمال تابع Softmax) را برای تمامی کلاس‌ها، در قالب یک نمودار میله‌ای¹⁴ در کنار تصویر مربوطه رسم کنید تا میزان اطمینان مدل برای هر دسته مشخص شود.

- با بررسی نمودارهای احتمال رسم‌شده، خطاهای مدل را تحلیل کنید. آیا شبکه در پیش‌بینی‌های نادرست خود با قطعیت (احتمال بالا) اشتباه کرده است یا احتمالات بین چند کلاس نزدیک به هم تقسیم شده‌اند؟

¹²Vanishing Gradient

¹³ Overfitting

¹⁴ Bar Chart

- با مشاهده خود تصاویر، به نظر شما چه شباهت‌های ظاهری و ویژگی‌های بصری مشترکی بین این سلول‌ها وجود داشته که باعث فریب خوردن و خطای شبکه‌ها شده است؟

بخش چهارم: جستجوی فضای ویژگی شبکه عصبی پیچشی (Feature Maps)

در این بخش قصد داریم نقشه‌های ویژگی (Feature Maps) را که حاصل پردازش تصویر توسط لایه‌های پیچشی هستند بصری سازی کنیم. این نقشه‌ها نمایانگر بخش‌هایی از تصویر هستند که شبکه به آن‌ها توجه بیشتری داشته است.

ابتدا یک تصویر تصادفی از مجموعه داده تست انتخاب می‌شود، سپس مدل را clip می‌کنیم. clip کردن مدل به این معناست که بخشی از شبکه عصبی را جدا کرده و فقط خروجی آن بخش را بررسی کنیم. این کار به ما اجازه می‌دهد تا نمایش میانی داده را در مراحل مختلف پردازش شبکه مشاهده کنیم و بفهمیم مدل در آن نقطه چه ویژگی‌هایی را استخراج کرده است. با این کار میتوان عملکرد داخلی شبکه را به صورت مرحله به مرحله تحلیل کرد و لایه‌های مختلف را از نظر قدرت تشخیص ویژگی‌ها بررسی نمود.

در این بخش شما باید تصویر انتخاب‌شده را به عنوان ورودی به اولین لایه پیچشی (Convolutional Layer) شبکه خود بدهید و خروجی میانی مدل clip شده را برای تمامی فیلترهای آن لایه نمایش دهید تا مشاهده کنید مدل در گام نخست به چه الگوهایی (مانند لبه‌ها یا بافت‌ها) حساسیت نشان داده است.

بخش پنجم: آزمون پایداری در برابر تغییرات مکانی (Spatial Invariance)

در این بخش قصد داریم پایداری و مقاومت دو مدل آموزش‌دیده را در برابر تغییرات هندسی مانند چرخش¹⁵ و انتقال مکانی¹⁶ ارزیابی کنیم. از آنجا که در تصاویر بیولوژیک و پزشکی، جهت‌گیری و محل قرارگیری سلول‌ها در کادر می‌تواند بسیار متفاوت باشد، بررسی اینکه آیا مدل دقت پیش‌بینی خود را تحت این شرایط حفظ می‌کند یا خیر، از اهمیت بالایی برخوردار است. این آزمایش تفاوت توانایی شبکه‌های CNN و MLP را در استخراج ویژگی‌های مستقل از مکان نشان می‌دهد.

برای انجام این آزمایش، ابتدا کدی بنویسید که برای تک‌تک کلاس‌های موجود، یک نمونه تصویر از داده‌های تست پیدا کند که هر دو مدل (CNN و MLP) پیش از این آن را به درستی طبقه‌بندی کرده باشند. سپس با استفاده از توابع پیش‌پردازش کتابخانه PyTorch، یک تبدیل هندسی شامل مقداری چرخش و انتقال مکانی بر روی این تصاویر انتخاب‌شده اعمال کنید.

¹⁵ Rotation

¹⁶ Translation

در مرحله بعد، توزیع احتمال پیش‌بینی هر دو مدل را برای تصاویر اصلی و تصاویر تغییر یافته، با استفاده از خروجی تابع Softmax محاسبه نمایید.

در نهایت، برای هر نمونه تصویر، موارد زیر نمایش داده شوند:

۱. تصویر اصلی همراه با برچسب واقعی

۲. نمودار میله‌ای احتمالات خروجی مدل MLP برای تصویر اصلی

۳. نمودار میله‌ای احتمالات خروجی مدل CNN برای تصویر اصلی

۴. تصویر تغییر یافته (پس از چرخش و انتقال)

۵. نمودار میله‌ای احتمالات خروجی مدل MLP برای تصویر تغییر یافته

۶. نمودار میله‌ای احتمالات خروجی مدل CNN برای تصویر تغییر یافته

- با مقایسه دقیق نمودارهای خروجی پیش و پس از اعمال تغییرات هندسی، عملکرد شبکه MLP و CNN را تحلیل کنید. کدام مدل در برابر چرخش و انتقال تصویر پایداری بیشتری از خود نشان داد و توانست پیش‌بینی صحیح خود را حفظ کند؟
- از نظر ساختاری و تئوری معماری شبکه‌ها، دلیل تفاوت عملکرد این دو مدل در چیست؟ چرا شبکه‌های عصبی پیچشی (CNN) ذاتاً توانایی بهتری در حفظ ثبات ویژگی‌ها و مقاومت در برابر انتقال مکانی (Spatial Invariance) دارند؟

بخش ششم: تحلیل حساسیت (Sensitivity Analysis) و بررسی Heatmap

در این بخش، قصد داریم با استفاده از تکنیک Occlusion Sensitivity، مشخص کنیم که مدل‌های آموزش‌دیده برای پیش‌بینی و دسته‌بندی صحیح تصاویر، دقیقاً به کدام نواحی توجه بیشتری می‌کنند. این روش به ما کمک می‌کند تا عملکرد درونی شبکه را شفاف‌تر کرده و بفهمیم آیا مدل واقعاً ویژگی‌های معنادار بیولوژیکی و ساختار سلول را یاد گرفته است یا صرفاً نویزهای بی‌ربط پس‌زمینه را حفظ کرده است.

برای پیاده‌سازی این ارزیابی، مراحل زیر را انجام دهید:

ابتدا یک پنجره مسدود کننده (Patch) با اندازه مشخص (برای مثال 8x8 پیکسل) در نظر بگیرید. سپس این پنجره را به صورت گام‌به‌گام در سراسر طول و عرض تصاویر تست (که ابعاد 64x64 دارند) حرکت دهید.

در هر گام، پیکسل‌های زیر این پنجره را درون تصویر پنهان یا مسدود کنید (بهترین روش برای این کار، جایگزین کردن مقادیر آن ناحیه با مقدار میانگین کل پیکسل‌های همان تصویر است). سپس این تصویر دستکاری‌شده را به عنوان ورودی به مدل‌های خود (MLP و CNN) بدهید و مقدار احتمال خروجی تابع Softmax را برای کلاس صحیح واقعی آن تصویر محاسبه کنید.

تفاوت و میزان افت احتمال خروجی تصویر اصلی نسبت به تصویر مسدودشده، نشان‌دهنده میزان اهمیت آن ناحیه از تصویر است؛ به این معنا که هرچه افت احتمال در یک گام بیشتر باشد، یعنی مسدود شدن آن بخش ضربه بزرگتری به تشخیص مدل زده و در نتیجه آن ناحیه برای تصمیم‌گیری مدل حیاتی‌تر بوده است. با ذخیره کردن این اختلافات احتمال برای تمامی نواحی، یک Heatmap برای هر دو مدل استخراج کنید. در نهایت، برای هر نمونه تصویر، موارد زیر نمایش داده شوند:

۱. تصویر اصلی همراه با نام کلاس واقعی

۲. Heatmap مدل MLP که روی تصویر اصلی هم‌پوشانی (Overlay) شده است.

۳. Heatmap مدل CNN که روی تصویر اصلی هم‌پوشانی شده است.

نکات پایانی

- دقت کنید که کد شما باید به نحوی زده شده باشد که نتایج قابلیت بازتولید داشته باشند.
- درباره هر بخش از مراحل پروژه می‌بایست علل استفاده یا عدم استفاده از هر الگوریتم، مزایا و معایب، عملکرد، فرا پارامترها و وضعیت خروجی‌ها را بطور دقیق مطالعه کنید. از این موضوعات در زمان تحویل پرسیده خواهد شد.
- پس از مطالعه کامل و دقیق صورت پروژه، در صورت وجود هرگونه ابهام یا سوال با طراحان پروژه در ارتباط باشید.
- نوت‌بوک کامل شده را با فرمت AI_CA5_LASTNAME_STUDENTID.ipynb نام گذاری کرده و در ای‌لرن بارگذاری کنید.
- توجه کنید این تمرین باید به صورت تک‌نفره انجام شود و پاسخ‌های ارائه شده باید نتیجه فعالیت فرد نویسنده باشد. در صورت مشاهده تقلب به همه افراد مشارکت‌کننده، نمره تمرین 100- و به استاد نیز گزارش می‌گردد. همچنین نوشته نشدن کدها توسط هوش مصنوعی نیز بررسی می‌شود!